05_phase_0_evidence

Phase 0 Evidence Log

Each task's acceptance check output is pasted below with a timestamp. This file is the rolled-up forensic record per synthesis spec §3.2 / Article XI §11.9.

P0-03 — HelixAgent submodule integration

Timestamp: 2026-05-04T20:56+03:00 **HelixAgent SHA committed:** `fe3f69e766077360d730da934e292f86646dadfd` **Total disk size:** 777 MB (under 1 GB threshold — no shallow-init follow-up needed)

Core libraries (all populated)

Library	Files/dirs	SHA
HelixLLM	25	`a385d9e3d1b74d8064c2b112ff773052a791eb20`
HelixMemory	15	`f1d55ea6cb297160790e4e21c472cf3a1626c42a`
HelixSpecifier	13	`8f83107842f50010c09f63064b649bcf21330bc3`
LLMsVerifier	266	`1d53ae3b72c77c1f27171c0677431c48d2d02bdd`

cli_agents

- **Total directory entries:** 60
- **Populated (≥2 files):** 47 (78%)
- **Phase 1 priority** `claude-code`**:** ✓ populated (`helix_agent/cli_agents/claude-code/`)
- **Empty (need Phase 2 sub-spec attention to fix HelixAgent's pin first):** 13
    - `aider, conduit, continue, HelixCode (recursive ref), kilo-code, kiro-cli (no upstream access), mobile-agent, ollama-code, opencode-cli, openhands, plandex, roo-code, superset`

Why empty: HelixAgent's recorded submodule pointers reference SHAs that no longer exist on the corresponding upstream remotes (e.g. force-push or branch deletion upstream). Examples: - `cli_agents/aider`: `5c536a29...` not found on aider's upstream - `cli_agents/plandex`: `9825d8d5...` not found on plandex's upstream - `cli_agents/kiro-cli`: repo `git@github.com:stark1tty/kiro-cli.git` inaccessible (no rights)

This is a **pre-existing HelixAgent issue**, not caused by this Phase 0 work. Per spec §1.3 N2 (HelixAgent rewrite is out of scope for this programme), the fix lives in HelixAgent's own governance: each affected agent's Phase 2 sub-spec will bump HelixAgent's submodule pointer to a SHA that exists upstream.

.gitmodules entry verified

```
[submodule "HelixAgent"]
    path = HelixAgent
    url = git@github.com:HelixDevelopment/HelixAgent.git
```

SSH URL per Constitution Rule 3.

Pre-commit secret scan

`git diff --cached` produced only `.gitmodules` + `HelixAgent` gitlink + `docs/improvements/05_phase_0_evidence.md` + `docs/improvements/PROGRESS.md`. Token-pattern grep returned zero matches.

Acceptance check status

Plan acceptance criterion	Result
`.gitmodules` has `[submodule "HelixAgent"]` SSH	✓
`helix_agent/{HelixLLM,HelixMemory,HelixSpecifier,LLMsVerifier}` exist + populated	✓ all four
`helix_agent/cli_agents/` count ≥35	✓ 60 entries (47 fully populated)
`helix_agent/cli_agents/claude-code` populated (Phase 1 priority)	✓
Pre-commit secret scan clean	✓
No third-party submodule modifications	✓ verified — no commits made inside any submodule
HelixAgent total size measured	✓ 777 MB

Outcome: Phase 1 (claude-code porting) is unblocked. The 13 unpopulated cli_agents are documented as deferred to Phase 2 sub-spec time when each affected agent's pin can be bumped in HelixAgent.

P0-04 — verify-llmsverifier-pin-parity.sh

Timestamp: 2026-05-04T21:30+03:00

Live pass-path output (Step 4.2)

```
FAIL: LLMsVerifier pin divergence
  submodules/llms_verifier  → 629c5bd5d141351270e72b6fb7359fa4b7881d7c
  helix_agent/LLMsVerifier → 1d53ae3b72c77c1f27171c0677431c48d2d02bdd

Resolution: pick the canonical SHA, bump the other to match, commit, push.
exit=1
```

Synthetic-divergence test (Step 4.4)

Because the live state was already divergent (exit=1), we used the canonical submodule's `HEAD^` (`1d53ae3b...` — which equals the transitive SHA) to temporarily bring the two into parity, proving the script correctly reports exit=0 on parity:

After checking out `PARENT_PARENT_SHA` (canonical→`1d53ae3b`, transitive→`1d53ae3b`):

```
OK: LLMsVerifier pin parity — both at 1d53ae3b72c77c1f27171c0677431c48d2d0
exit=0 (expected — script correctly detects parity)
```

After restoring canonical to `PARENT_SHA` (`629c5bd5`):

```
FAIL: LLMsVerifier pin divergence
  submodules/llms_verifier  → 629c5bd5d141351270e72b6fb7359fa4b7881d7c
  helix_agent/LLMsVerifier → 1d53ae3b72c77c1f27171c0677431c48d2d02bdd

Resolution: pick the canonical SHA, bump the other to match, commit, push.
exit=1  (matches Step 4.2 — back to original live state)
```

Restore verified: `git -C submodules/llms_verifier rev-parse HEAD` returns `629c5bd5d141351270e72b6fb7359fa4b7881d7c` — correct.

Live divergence status

Pins diverge — see PROGRESS.md parking lot for resolution.

- `submodules/llms_verifier` → `629c5bd5d141351270e72b6fb7359fa4b7881d7c`
- `helix_agent/LLMsVerifier` → `1d53ae3b72c77c1f27171c0677431c48d2d02bdd`

The canonical pin is one commit ahead of the transitive (HelixAgent) pin. Resolution deferred per spec §1.3 N2.

P0-05 — API-key migration from ../helix_agent/.env

Timestamp: 2026-05-04T21:15:12+03:00

Source: `-rw------- milosvasic milosvasic 7603 /run/media/milosvasic/DATA4TB/Projects/helix_agent/.env`

Destination: `-rw------- milosvasic milosvasic 7603 /run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/.env`

Key count: 109

Key set diff (source vs destination):

(empty diff = identical key set)

In git index — precise exact-match check: `helix_code/helix_code/.env` is **NOT** tracked by git (`git ls-files --error-unmatch helix_code/helix_code/.env`

exits 1). The earlier substring-grep count of 2 reflected unrelated tracked files (`helix_code/helix_code/.env.example` and `helix_code/helix_code/.env.full-test`), not the secret-bearing file.

P0-06 — .gitignore hardening

Timestamp: 2026-05-04T21:22:51+03:00

Root .gitignore tail:

```
# Allow the e2e challenge testing framework but ignore test results
!helix_code/tests/e2e/challenges/
helix_code/tests/e2e/challenges/test-results/
helix_code/tests/e2e/challenges/.DS_Store


reports/demos/

# === Secret hygiene (CONST-042) — added P0-06 ===
.env.*
!.env.example
*.pem
*.key
*.crt
id_rsa
id_rsa.pub
id_ed25519
id_ed25519.pub
helix.security.json
# === END Secret hygiene ===
```

Inner .gitignore tail:

```


# nyc test coverage
.nyc_output

# Dependency directories (if using Go modules, comment out vendor/ above)
# vendor/

# End of https://www.toptal.com/developers/gitignore/api/go,vim,emacs,visu
cli

# === Secret hygiene (CONST-042) — added P0-06 ===
.env
.env.*
!.env.example
id_rsa
id_rsa.pub
id_ed25519
id_ed25519.pub
helix.security.json
# === END Secret hygiene ===
```

Verifications: - helix_code/.env is ignored: YES - helix_code/.env.example is NOT ignored: YES (good) - Tracked credential files (pre-existing CONST-042 violations, all committed before this task): **three files** are in the git index: - `helix_code/test/workers/ssh_keys/id_rsa` — SSH private key labelled `helixcode-test` - `helix_code/test/workers/ssh_keys/id_rsa.pub` — corresponding public key - `helix.security.json` — root-level security credential file (5929 bytes, executable)

All three were committed before this programme began. The CONST-042 `.gitignore` blocks prevent any NEW untracked instances of these patterns from being accidentally added. Proper remediation — key/credential rotation, `git rm --cached` to remove from index, regeneration of any derived secrets, and historical-leak documentation — is deferred to **T08** (`scripts/scan-secrets.sh`). The planted-secret test in T08 will fail on the live tree due to these three files, triggering tracked remediation through the standard scan-secrets workflow.

- **Asymmetric coverage between root and inner `.gitignore` CONST-042 blocks** is intentional and correct:
  - The **root block** adds 10 patterns: omits `.env.local` (pre-existing at root `.gitignore` line 5) and omits `.env` at block level (pre-existing at line 4; P0-06 polish adds it back into the block to make the block self-contained).
  - The **inner block** adds 8 patterns: omits `*.pem`, `*.key`, `*.crt` (pre-existing at inner `.gitignore` lines 85–87); omits `.env.local` (pre-existing at inner `.gitignore` line 44).
  - **Effective combined coverage**: all 12 canonical secret-file patterns are protected at both the root and inner levels — the asymmetry reflects de-duplication of already-existing lines, not a coverage gap.

P0-07 — .env.example refresh

Timestamp: 2026-05-04T21:36:23+03:00

Key parity vs ../helix_agent/.env: OK (identical)

Real values present: 0 (must be 0)

Total keys: 109

Verification commands and output:

```
$ grep -oE '^[A-Z_]+=' ../helix_agent/.env | sort -u > /tmp/p0-07-canonica
$ wc -l /tmp/p0-07-canonical-keys.txt
109 /tmp/p0-07-canonical-keys.txt

$ diff <(grep -oE '^[A-Z_]+=' ../helix_agent/.env | sort -u) \
       <(grep -oE '^[A-Z_]+=' helix_code/.env.example | sort -u)
key-diff-exit=0

$ result=$(grep -E '^[A-Z_]+=[^<]' helix_code/.env.example | grep -vE '=$'
$ [ -z "$result" ] && echo "VERIFIED: zero real values present in .env.exa
VERIFIED: zero real values present in .env.example
real-value-count=0
```

CONST-042 status: COMPLIANT — every entry in `.env.example` is either a comment, blank line, or `KEY=<REDACTED>`.

File format: 7-line header block + 1 blank line + 109 KEY=<REDACTED> lines = 117 lines total.

P0-08 — scan-secrets.sh + planted-secret test

Timestamp: 2026-05-04T22:15+03:00

Test harness output (Step 8.4)

```
TEST 1: clean directory → expect exit 0
  PASS
TEST 2: planted secret → expect non-zero exit
  PASS — scanner detected the planted secret

Results: 2 passed, 0 failed
exit=0
```

Live-tree run (Step 8.5)

Live exit code: **1** (expected — known credential files in tracked tree per T06 polish evidence; NOT a script defect)

Filenames flagged by the scanner (file:line only — values redacted per CONST-042):

```
./challenges/panoptic/docs/SECURITY.md:462:
./challenges/panoptic/tests/security/panoptic_test.go:422:
./challenges/panoptic/tests/security/panoptic_test.go:423:
./challenges/panoptic/tests/security/panoptic_test.go:454:
./challenges/pkg/env/redact_test.go:37:
./challenges/pkg/env/redact_test.go:48:
./challenges/pkg/logging/redacting_logger_test.go:55:
./docs/COMPLETE_CLI_REFERENCE.md:908:
./docs/superpowers/plans/2026-05-04-phase-0-foundation-cleanup.md:833:
./docs/troubleshooting/guide.md:649:
./.env:188:
./.env:20:
./.env:4:
./.env:6:
./.env:9:
./helix_code/.env:118:
./helix_code/.env:119:
./helix_code/.env:186:
./helix_code/.env:29:
./helix_code/.env:41:
./helix_code/.env:43:
./helix_code/.env:46:
./helix_code/.env:47:
./helix_code/.env:53:
./helix_code/internal/llm/vertexai_provider_test.go:25:
./helix_code/internal/worker/ssh_pool_test.go:569:
./helix_code/tests/e2e/test_bank/performance_security_tests.go:1073:
./helix_code/test/workers/ssh_keys/id_rsa:1:
./helix_qa/pkg/llm/google_test.go:334:
./security/pkg/securestorage/securestorage_test.go:129:
```

Findings

Tracked files with matches: | File | Tracked | Nature | |—|—|—| | `helix_code/test/workers/ssh_keys/id_rsa` | YES | Pre-existing tracked SSH private key (T06 known credential #1) | | `helix.security.json` | YES | Pre-existing tracked credential file (T06 known credential #2) | | `helix_code/test/workers/ssh_keys/id_rsa.pub` | YES (no match) | Public key — does not match any pattern | | `docs/COMPLETE_CLI_REFERENCE.md:908` | YES | Example/doc text: `sk-ant-your-anthropic-key` (placeholder, not a real key) | | `docs/superpowers/plans/...foundation-cleanup.md:833` | YES | Planted-secret TDD test example: `sk-FAKE0123456789...` (fake) | | `docs/troubleshooting/guide.md:649` | YES | Documentation snippet: `-----BEGIN OPENSSH PRIVATE KEY-----` (illustrative, incomplete) | | `helix_code/internal/llm/vertexai_provider_test.go:25` | YES | Unit-test fixture: embedded fake RSA key block (test data, not rotatable) | | `helix_code/internal/worker/ssh_pool_test.go:569` | YES | Unit-test stub: partial `-----BEGIN OPENSSH PRIVATE KEY-----` header only | | `helix_code/tests/e2e/test_bank/performance_security_tests.go:1073` | YES | Test fixture: `sk-1234567890abcdef` (clearly fake) |

Not-tracked files flagged (untracked working-tree files — not a commit risk): - `.env`, `helix_code/.env`: the real secret-bearing env files (correctly untracked per P0-06 gitignore) - `challenges/panoptic/...`, `helix_qa/...`, `security/...`: submodule working-tree files, not tracked at root

The 3 pre-existing tracked credentials from T06 polish: - `helix_code/test/workers/ssh_keys/id_rsa` — correctly detected by scanner (pattern: `-----BEGIN ... PRIVATE KEY-----`) - `helix_code/test/workers/ssh_keys/id_rsa.pub` — public key, no secret pattern matches (expected) - `helix.security.json` — NOT detected in this run (see note below)

Note on `helix.security.json`: The scanner's `SCAN_TARGET="."` run did not flag `helix.security.json` in the output above. The file's content does not match any of the scanner's current patterns (it's a JSON credential file that likely uses JWTs or other formats not in the regex set). The file IS blocked from future commits via `.gitignore` (P0-06) and will be addressed in P0-T08.5 remediation. The scanner's pattern set can be extended in P0-T08.5 to cover JWT/JSON credential formats.

Additional tracked files with fake/doc patterns (`docs/*.md` and test fixtures): these are false-positive-in-intent — the patterns are doc examples and test fixtures, not real rotatable secrets. They will be reviewed during P0-T08.5 to determine whether to add per-file exclusions or restructure test data. No immediate action required.

P0-08.7 — Port SonarQube + Snyk security scan integration (through Containers)

Timestamp: 2026-05-04T22:30+03:00 **Commits:** `1d728de` + `2494bc8` + `e29e2f6` + `16a4490` + (this commit) **Branch:** main

Sub-commit 1 — Compose files + configs (`1d728de`)

Files created: - `helix_code/docker/security/sonarqube/docker-compose.yml` — adapted from HelixAgent; container names changed from `helixagent-*` to `helixcode-`

*; subnet changed to 172.21.0.0/16 - helix_code/docker/security/sonarqube/sonar-project.properties — project key/name/version use ${SONARQUBE_PROJECT_KEY} etc. - helix_code/docker/security/snyk/docker-compose.yml — adapted; uses ${SNYK_TOKEN:-} - helix_code/docker/security/snyk/Dockerfile — adapted; references HelixCode project - helix_code/sonar-project.properties — root-level; uses ${SONARQUBE_PROJECT_KEY} env-var - helix_code/.snyk — root-level Snyk policy (v1.25.0)

Credential scan result: No credentials found - OK

Sub-commit 2 — Master orchestrator script (2494bc8)

- helix_code/scripts/security-scan.sh — supports snyk|sonarqube|trivy|gosec|grype|kics|semgrep|all
- Loads credentials from helix_code/.env (gitignored)
- Reports to reports/security/<scanner>-<timestamp>.<ext>
- --help dry run verified:

```
$ ./scripts/security-scan.sh --help
Usage:
  ./scripts/security-scan.sh [scanner] [options]

Scanners:
  snyk        - Snyk vulnerability scanner (requires SNYK_TOKEN)
  sonarqube   - SonarQube code quality and security analysis
  ...
exit 0
```

Sub-commit 3 — Makefile targets (e29e2f6)

Inner Makefile (helix_code/Makefile) targets added: security-scan, security-scan-snyk, security-scan-sonarqube, security-scan-trivy, security-scan-gosec, security-scan-grype, security-scan-kics, security-scan-semgrep, security-scan-all, deps-scan, secrets-scan, scan-start-sonar, scan-stop

Root Makefile targets added: scan-sonarqube, scan-snyk, scan-all, scan-gosec, scan-trivy, scan-secrets

Dry-run verification:

```
$ make -n scan-sonarqube
make -C HelixCode security-scan-sonarqube
./scripts/security-scan.sh sonarqube
```

Sub-commit 4 — containers BootManager wiring (16a4490)

- helix_code/cmd/security_scan/main.go (~170 lines) wires:
  - digital.vasic.containers/pkg/runtime.AutoDetect(ctx) for runtime detection
  - digital.vasic.containers/pkg/endpoint.NewEndpoint() builder for SonarQube + Snyk endpoints

    - digital.vasic.containers/pkg/health.NewDefaultChecker() with HTTP health on :9000/api/system/status
    - digital.vasic.containers/pkg/boot.NewBootManager().BootAll(ctx) for lifecycle management
    - Supports -action=start|stop|status

Build verification:

```
$ go build ./cmd/security_scan/...
# exit 0 — binary compiles
```

scripts/security-scan.sh start_sonarqube() updated to call go run ./cmd/security_scan when Go is available; falls back to direct compose otherwise.

Sub-commit 5 — Challenges + evidence (this commit)

Challenges created: - helix_code/tests/e2e/challenges/sonarqube/run.sh — 8 sections, 33 tests (config-correctness only) - helix_code/tests/e2e/challenges/sonarqube/expected.json - helix_code/tests/e2e/challenges/snyk/run.sh — 7 sections, 26 tests (config-correctness only) - helix_code/tests/e2e/challenges/snyk/expected.json

Challenge run output (both 100% PASS):

```
==========================================
  SonarQube Security Scanning Challenge
==========================================
[PASS] Root sonar-project.properties exists
[PASS] SonarQube docker-compose.yml exists
[PASS] SONAR_TOKEN uses env-var reference (not hardcoded)
[PASS] sonar.projectKey uses env-var reference in root properties
[PASS] No HelixAgent-specific references in HelixCode configs
[PASS] SonarQube service defined
[PASS] PostgreSQL service defined
[PASS] SonarQube uses pinned community edition image
[PASS] Memory limits configured
[PASS] SonarQube health check uses /api/system/status
[PASS] Security network defined
[PASS] security-scan.sh --help shows sonarqube mode
[PASS] cmd/security_scan imports containers BootManager
[PASS] cmd/security_scan uses runtime.AutoDetect
Results: 33/33 passed, 0 failed

==========================================
  Snyk Security Scanning Challenge
==========================================
[PASS] Snyk docker-compose.yml exists
[PASS] SNYK_TOKEN uses env-var reference (not hardcoded)
[PASS] No HelixAgent-specific container names
[PASS] snyk-deps service defined
[PASS] snyk-full service defined
[PASS] Dockerfile uses official snyk/snyk-cli base image
[PASS] .snyk has policy version
[PASS] security-scan.sh reads SNYK_TOKEN from env
```

```
[PASS] cmd/security_scan imports containers BootManager
[PASS] cmd/security_scan handles snyk scanner
Results: 26/26 passed, 0 failed
```

CREDENTIAL ROTATION NOTE — MANDATORY before live scans

This task does NOT run live scans. The original `helix.security.json` credentials (SonarQube token, Snyk token, project_key, organization) were committed and are considered compromised (see P0-T08.5). The user MUST rotate these before any live scan can succeed:

1. **SonarQube token**: generate a new API token in SonarQube UI → set `SONAR_TOKEN=<new>` in `helix_code/.env`
2. **SonarQube project key**: choose a new key → set `SONARQUBE_PROJECT_KEY=<new>` in `helix_code/.env`
3. **Snyk token**: generate a new token at snyk.io → set `SNYK_TOKEN=<new>` in `helix_code/.env`
4. **Snyk organization**: set `SNYK_ORG=<new_org_id>` in `helix_code/.env`

After rotation, live scan can be invoked with:

```
make scan-sonarqube    # from root
make scan-snyk         # from root
```

scan-secrets.sh verification

```
$ bash scripts/scan-secrets.sh helix_code/docker/security helix_code/sonar
OK: no credential patterns found
exit code: 0
```

Remote convergence (all 5 commits)

All 3 distinct remotes (github, gitlab, upstream) converged on each sub-commit SHA. Final HEAD after sub-commit 5 (this evidence): see PROGRESS.md.

Script correctness verdict: The scanner is operating correctly on real data. It detects the known tracked private key (`id_rsa`) and real api-key patterns in the live `.env` files. Live-run exit=1 is correct given the presence of pre-existing tracked credentials.

P0-T08.7 (fix-it) — T08.7 code-quality review findings

Timestamp: 2026-05-04T22:40+03:00 **Fixes applied:** Critical 1, Critical 2, Important 3, Important 4, Important 5, Important 6 (TODO), Important 7

Critical 1 — Hardcoded DB credentials

`helix_code/docker/security/sonarqube/docker-compose.yml` — replaced hardcoded `sonar` values:

```
# Before:
SONAR_JDBC_USERNAME: sonar
SONAR_JDBC_PASSWORD: sonar
POSTGRES_USER: sonar
```

```
POSTGRES_PASSWORD: sonar

# After:
SONAR_JDBC_USERNAME: ${SONARQUBE_DB_USER:-sonar}
SONAR_JDBC_PASSWORD: ${SONARQUBE_DB_PASSWORD:-sonar}
POSTGRES_USER: ${SONARQUBE_DB_USER:-sonar}
POSTGRES_PASSWORD: ${SONARQUBE_DB_PASSWORD:-sonar}
```

Verification:

```
$ grep -nE "PASSWORD: sonar$|PASSWORD: \"sonar\"$" helix_code/docker/secur
# (empty — PASS)

$ grep -nE "PASSWORD:.*sonar" helix_code/docker/security/sonarqube/docker-
21:      SONAR_JDBC_PASSWORD: ${SONARQUBE_DB_PASSWORD:-sonar}
56:      POSTGRES_PASSWORD: ${SONARQUBE_DB_PASSWORD:-sonar}
# env-var form — PASS
```

Critical 2 — Stale TODO removed

`helix_code/scripts/security-scan.sh` lines 31-34 removed. Original stale text:

```
# TODO(P0-T08.7/4): replace docker-compose invocations below with containe
#   go run ./cmd/security_scan -scanner=sonarqube
# For now, direct compose calls are used as an MVP; the containers BootMan
# is deferred to Sub-commit 4 / Phase 3.
```

Replaced with single-line accurate status note. Sub-commit 4 (`16a4490`) had already landed.

Verification:

```
$ grep -nE "deferred to Sub-commit 4|For now, direct compose calls" helix_
# (empty — PASS)
```

Important 3 — set -euo pipefail

```
$ head -40 helix_code/scripts/security-scan.sh | grep -nE "^set -"
36:set -euo pipefail
# PASS

$ helix_code/scripts/security-scan.sh --help > /dev/null 2>&1; echo "exit=
exit=0
# PASS — no unbound variable errors on --help path
```

Important 4 — stop action returns explicit error

`helix_code/cmd/security_scan/main.go` — both `handleSonarQube` and `handleSnyk` stop cases now return `fmt.Errorf(...)`. Flag description updated to `"start|status (stop is not yet implemented)"`.

Verification:

```
$ go build ./cmd/security_scan/...
# exit=0 — PASS
```

```
$ go run ./cmd/security_scan/main.go -action=stop -scanner=sonarqube 2>&1
2026/05/04 22:38:15 security-scan: detected runtime: podman
2026/05/04 22:38:15 security-scan: sonarqube stop failed: stop action not
exit status 1
# exits non-zero with explicit message — PASS
```

Important 5 — :latest image tags pinned

All 5 container-fallback image tags in `helix_code/scripts/security-scan.sh` pinned: - `securego/gosec:latest` → `securego/gosec:2.21.4` - `aquasec/trivy:latest` → `aquasec/trivy:0.55.2` - `anchore/grype:latest` → `anchore/grype:v0.86.0` - `checkmarx/kics:latest` → `checkmarx/kics:v2.1.4` - `returntocorp/semgrep:latest` → `returntocorp/semgrep:1.93.0`

Verification:

```
$ grep -nE ":latest" helix_code/scripts/security-scan.sh
# (empty — PASS)
```

Important 6 — Refactor deferred with TODO

Added TODO comment at top of `security-scan.sh` (line 6):

```
#
        TODO(P3-refactor): this file handles 7+ scanners in ~580
        lines; split into
# per-scanner sourced modules under scripts/scanners/ in a future
        Phase 3 task.
```

Important 7 — reports/security/ gitignored

Added to `helix_code/.gitignore`:

```
# === Security reports (never commit scanner output) ===
reports/security/
reports/security-cache/
# === END Security reports ===
```

Verification:

```
$ git check-ignore helix_code/reports/security/test.json; echo "exit=$?"
helix_code/reports/security/test.json
exit=0
# PASS
```

scan-secrets clean

```
$ ./scripts/scan-secrets.sh
OK: no credential patterns found in .
exit=0
```

P0-09 — pre-push hook + installer + setup.sh wiring

Timestamp: 2026-05-04T22:43:12+03:00

Hook source: `-rwxr-xr-x scripts/git_hooks/pre-push`

Hook installed: `-rwxr-xr-x .git/hooks/pre-push`

Installer output (first run — 1 hook installed):

```
  installed: .git/hooks/pre-push
OK: 1 hook(s) installed/updated under .git/hooks
```

Installer output (second run — idempotency verified):

```
OK: 0 hook(s) installed/updated under .git/hooks
```

setup.sh hook-install line:

```
./scripts/install-git-hooks.sh
```

Force-block logic verified by inspection (10/10 cases PASS):

All three force-flag variants match correctly in the hook's case statement:

Test cmdline	Expected is_force	Result
`git push origin main --force`	1	PASS
`git push origin main --force`	1	PASS
`git push --force origin main`	1	PASS
`git push -f origin main`	1	PASS
`git push origin main -f`	1	PASS
`git push --force-with-lease origin main`	1	PASS
`git push --force-with-lease`	1	PASS
`git push origin main`	0	PASS
`git push origin main --no-force`	0	PASS
`git push upstream main`	0	PASS

HELIX_FORCE_PUSH_APPROVED=1 bypass verified: - Force push WITHOUT approval → BLOCKED (exit 1) - Force push WITH `HELIX_FORCE_PUSH_APPROVED=1` → ALLOWED (exit 0)

Non-force direct invocation:

```
$ bash scripts/git_hooks/pre-push origin git@github.com:HelixDevelopment/H
exit=0
```

Platform degradation: When `/proc/$PPID/cmdline` is not readable (macOS/BSD), the hook warns but exits 0 — it does not block. CONST-043 constitutional clause is the actual contract.

scan-secrets post-install verification:

```
$ ./scripts/scan-secrets.sh
OK: no credential patterns found in .
exit=0
```

Real push (this commit) succeeds: indicates non-force pushes are not blocked by the installed hook.

CONST-042 compliance: scan-secrets clean (above). **CONST-043 compliance:** pre-push hook is installed and blocks –force / -f / –force-with-lease without HELIX_FORCE_PUSH_APPROVED=1.

P0-10 — helix_code/ inner Go-app governance triplet

Timestamp: 2026-05-04T22:55+03:00 **Branch:** main

Files created

File	Size	Path
CONSTITUTION.md	5622 bytes	helix_code/ helix_code/ CONSTITUTION.md
CLAUDE.md	6204 bytes	helix_code/ helix_code/ CLAUDE.md
AGENTS.md	4787 bytes	helix_code/ helix_code/ AGENTS.md

Anchor verification

Each file was verified to contain all three constitutional anchors:

```
$ grep -c "11\.9\|tests do execute" helix_code/CONSTITUTION.md helix_code/
helix_code/CONSTITUTION.md:3
helix_code/CLAUDE.md:2
helix_code/AGENTS.md:2

$ grep -c "CONST-042\|CONST-043" helix_code/CONSTITUTION.md helix_code/CLA
helix_code/CONSTITUTION.md:2
helix_code/CLAUDE.md:3
helix_code/AGENTS.md:4
```

All files contain: - Article XI §11.9 anti-bluff anchor (verbatim user mandate + operative rule) ✓ - CONST-042 (No-Secret-Leak) ✓ - CONST-043 (No-Force-Push) ✓

ADDENDA markers verified

```
$ grep -c "BEGIN: REPO-SPECIFIC ADDENDA\|END: REPO-SPECIFIC ADDENDA" \
    helix_code/CONSTITUTION.md helix_code/CLAUDE.md helix_code/AGENTS.md
helix_code/CONSTITUTION.md:2
helix_code/CLAUDE.md:2
helix_code/AGENTS.md:2
```

All three files have `<!-- BEGIN: REPO-SPECIFIC ADDENDA -->` / `<!-- END: REPO-SPECIFIC ADDENDA -->` delimiters. ✓

Synthesis spec references verified

```
$ grep -c "2026-05-04-cli-agent-fusion-synthesis-design" \
    helix_code/CONSTITUTION.md helix_code/CLAUDE.md helix_code/AGENTS.md
helix_code/CONSTITUTION.md:1
helix_code/CLAUDE.md:2
helix_code/AGENTS.md:2
```

All three reference `../docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md`. ✓

Secret scan

```
$ for f in helix_code/CLAUDE.md helix_code/AGENTS.md helix_code/CONSTITUTI
    bash scripts/scan-secrets.sh "$f"; echo "[$f] exit=$?"
  done
OK: no credential patterns found in helix_code/CLAUDE.md
[helix_code/CLAUDE.md] exit=0
OK: no credential patterns found in helix_code/AGENTS.md
[helix_code/AGENTS.md] exit=0
OK: no credential patterns found in helix_code/CONSTITUTION.md
[helix_code/CONSTITUTION.md] exit=0
```

CONST-042 compliance: ✓ clean

Acceptance checklist

Criterion	Result
All three files exist at `helix_code/{CLAUDE,AGENTS,CONSTITUTION}.md`	✓
Each file contains Article XI §11.9 verbatim	✓
Each file contains CONST-042 (No-Secret-Leak)	✓
Each file contains CONST-043 (No-Force-Push)	✓
Each file has REPO-SPECIFIC ADDENDA delimiters	✓
Each file references synthesis spec	✓

Criterion	Result
No secrets present (scan-secrets clean)	✓
No third-party submodule modifications	✓

P0-11 — Article XII added to root CONSTITUTION.md

Timestamp: 2026-05-04T21:30:00+03:00

Anchors added: - Article XII §12.1 (CONST-042) — No-Secret-Leak - Article XII §12.2 (CONST-043) — No-Force-Push

Verification:

```
464:## Article XII — Repository Safety
466:### §12.1 (CONST-042) — No-Secret-Leak
477:### §12.2 (CONST-043) — No-Force-Push
486:**Cascade requirement:** Same as §12.1 — verbatim, every owned-by-us r
```

Existing CONST-041 (MCP) intact:

```
436:## CONST-041: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Int
```

Total CONST-NNN entries (CONST-001 through CONST-041): 41 (unchanged)

Insertion point: Between line 462 (`---` separator after CONST-041) and `## Amendment Process` section, at line 464.

P0-12 — root sister-file cascade (CLAUDE/AGENTS/CRUSH/QWEN)

Timestamp: 2026-05-04T23:15:00+03:00

Anchor presence after cascade:

```
  CLAUDE.md    anti-bluff=3  CONST-042=1  CONST-043=1
  AGENTS.md    anti-bluff=2  CONST-042=1  CONST-043=1
  CRUSH.md     anti-bluff=2  CONST-042=1  CONST-043=1
  QWEN.md      anti-bluff=2  CONST-042=1  CONST-043=1
```

Legitimate CONST-041 cross-reference preserved in AGENTS.md:

```
425:**Constitutional Impact**: Violates CONST-041 (MCP/LSP/ACP/Embedding/R
```

Secret scan (four files):

```
OK: no credential patterns found in CLAUDE.md
OK: no credential patterns found in AGENTS.md
OK: no credential patterns found in CRUSH.md
OK: no credential patterns found in QWEN.md
all-exit=0
```

Submodule audit: No third-party submodule modifications — HelixAgent, Containers, Challenges, HelixQA, Security, LLMsVerifier all had zero commits in last 30 minutes.

Edit strategy: - `CLAUDE.md` + `AGENTS.md`: existing anti-bluff section replaced with canonical `## Constitutional anchors` block (which includes §11.9 + CONST-042 + CONST-043). Non-canonical ⚠️ inline format superseded by structured heading block. - `CRUSH.md` + `QWEN.md`: full canonical anchor block inserted after title/intro paragraph (previously missing all anchors).

Acceptance checklist: | Criterion | Result | |—|—| | CLAUDE.md has anti-bluff §11.9 (count ≥1) | ✓ count=3 | | CLAUDE.md has CONST-042 (count ≥1) | ✓ count=1 | | CLAUDE.md has CONST-043 (count ≥1) | ✓ count=1 | | AGENTS.md has anti-bluff §11.9 (count ≥1) | ✓ count=2 | | AGENTS.md has CONST-042 (count ≥1) | ✓ count=1 | | AGENTS.md has CONST-043 (count ≥1) | ✓ count=1 | | CRUSH.md has anti-bluff §11.9 (count ≥1) | ✓ count=2 | | CRUSH.md has CONST-042 (count ≥1) | ✓ count=1 | | CRUSH.md has CONST-043 (count ≥1) | ✓ count=1 | | QWEN.md has anti-bluff §11.9 (count ≥1) | ✓ count=2 | | QWEN.md has CONST-042 (count ≥1) | ✓ count=1 | | QWEN.md has CONST-043 (count ≥1) | ✓ count=1 | | AGENTS.md CONST-041 (MCP) cross-ref preserved | ✓ line 425 | | Secret scan clean (4 files) | ✓ all exit=0 | | No third-party submodule modifications | ✓ |

P0-13 — root CLAUDE.md §3.2 bluff fix

Timestamp: 2026-05-04T23:20:29+03:00

Corrected line (CLAUDE.md:112):

```
├── helix_code/       ← TRACKED SUBDIRECTORY (NOT a submodule — meta-repo's
```

No remaining mislabel: `grep -nE "HelixCode.*SUBMODULE" CLAUDE.md` returns only this corrected line (with “NOT a submodule” qualifier present).

P0-14 — governance cascade across owned-by-us submodules

Timestamp: 2026-05-04T23:50:00+03:00

Verifier extended patterns: CONST-042, CONST-043 added to MANDATORY_PATTERNS in `scripts/verify-governance-cascade.sh`.

Root files fixed: Added “The bar for shipping is not…” to root CLAUDE.md and AGENTS.md (§11.9 operative rule), which then cascaded to all submodules.

Cascade verifier output (final):

```
GOVERNANCE_CASCADE: PASSED
exit=0
```

Owned-by-us submodule SHAs after cascade:

Submodule	SHA	Branch
helix_qa	ecebe9a	main
Challenges	53d47c8	main

Submodule	SHA	Branch
containers	6736040	main
Security	e7c09c1	main
submodules/llms_verifier	d473231d	main
submodules/doc_processor	764a9a9	master
submodules/llm_orchestrator	c2b04ad	master
submodules/llm_provider	afe0ac5	master
submodules/vision_engine	9a35a9f	master
HelixAgent	9a19ac12	main
helix_agent/HelixLLM	4a412c7	main
helix_agent/HelixMemory	e464257	main
helix_agent/HelixSpecifier	f1f9927	main

Excluded from cascade (third-party): `helix_agent/cli_agents/*`, `Example_Projects/*`, `dependencies/{Ollama,LLama_CPP,HuggingFace_Hub}`, `awesome-ai-memory`, `github_pages_website`, `Assets`.

Scripts modified: - `scripts/verify-governance-cascade.sh`: Added CONST-042, CONST-043 to MANDATORY_PATTERNS; excluded assets/github_pages_website from `is_helixcode_owned`; added HelixAgent nested submodules to ownership list. - `scripts/propagate-governance.sh`: Added `is_owned()` guard to prevent cascading into third-party submodules; added explicit branch-aware push with detached-HEAD protection.

P0-15 — Makefile verify-foundation gate

Timestamp: 2026-05-04T00:00+03:00

Targets added to root Makefile

Target	Description
`verify-llmsverifier-pin-parity`	Wraps `scripts/verify-llmsverifier-pin-parity.sh`
`verify-governance-cascade`	Wraps `scripts/verify-governance-cascade.sh`
`bluff-detector`	Stub-tolerant: runs `scripts/bluff-detector.sh` if present; otherwise prints skip message and exits 0. Phase 4 deliverable.
`scan-secrets-root`	Wraps root-level `scripts/scan-secrets.sh` (whole-repo scan, as distinct from `scan-secrets` which delegates to inner HelixCode scanner)
`verify-foundation`	

Target	Description
	Composite gate: `no-silent-skips-warn scan-secrets-root verify-llmsverifier-pin-parity verify-governance-cascade bluff-detector`

`ci-validate-all` **updated:** now depends on `verify-foundation` in addition to `no-silent-skips-warn` and `demo-all-warn`.

Note on `scan-secrets`**:** already existed (added by P0-T08.7), delegating to `helix_code/scripts/scan-secrets.sh`. Not redeclared. `verify-foundation` uses `scan-secrets-root` instead to invoke the whole-repo root scanner.

Individual gate verification

```
$ make verify-llmsverifier-pin-parity 2>&1 | tail -5; echo "exit=$?"
FAIL: LLMsVerifier pin divergence
  submodules/llms_verifier  → d473231d27196e2151405f37936151a386b590e3
  helix_agent/LLMsVerifier → 1d53ae3b72c77c1f27171c0677431c48d2d02bdd

Resolution: pick the canonical SHA, bump the other to match, commit, push.
exit=1    ← expected (known divergence, parking-lot item)

$ make verify-governance-cascade 2>&1 | tail -2; echo "exit=$?"
GOVERNANCE_CASCADE: PASSED
exit=0    ← expected

$ make scan-secrets-root 2>&1 | tail -2; echo "exit=$?"
OK: no credential patterns found in .
exit=0    ← expected

$ make bluff-detector 2>&1; echo "exit=$?"
bluff-detector.sh not yet implemented (Phase 4 deliverable); skipping
exit=0    ← expected
```

`make verify-foundation` output (last 15 lines)

```
(warn-only mode — set NO_SILENT_SKIPS_WARN_ONLY=0 to fail the build)
OK: no credential patterns found in .
FAIL: LLMsVerifier pin divergence
  submodules/llms_verifier  → d473231d27196e2151405f37936151a386b590e3
  helix_agent/LLMsVerifier → 1d53ae3b72c77c1f27171c0677431c48d2d02bdd

Resolution: pick the canonical SHA, bump the other to match, commit, push.
make: *** [Makefile:54: verify-llmsverifier-pin-parity] Error 1
make-exit=2
```

Expected behaviour: exits 2 (make error propagation from prerequisite exit 1). The divergence is the documented LLMsVerifier dual-pin parking-lot item. This is NOT a defect in P0-15; P0-16 close-out depends on resolution.

Acceptance checklist

Criterion	Result
`verify-llmsverifier-pin-parity` target wired	✓
`verify-governance-cascade` target wired	✓
`bluff-detector` target wired (stub-tolerant)	✓
`scan-secrets-root` target wired (root scanner)	✓
`verify-foundation` composite target wired	✓
`.PHONY` includes all new targets	✓
`scan-secrets` NOT redeclared (pre-existing)	✓ — already present from P0-T08.7
`ci-validate-all` updated to depend on `verify-foundation`	✓
Each individual gate runs correctly	✓ (see above)
`verify-foundation` exits non-zero on known parity divergence	✓ exit=2

P0-16 — Phase 0 close-out

Timestamp: 2026-05-05T00:05+03:00

Diagrams regenerated to `docs/improvements/06_diagrams_real/`**:**

```
total 348
-rw-r--r-- 1 milosvasic milosvasic 105650 May  4 23:59 dependency_graph.pn
-rw-r--r-- 1 milosvasic milosvasic 129320 May  4 23:59 feature_gap_matrix.
-rw-r--r-- 1 milosvasic milosvasic  45270 May  4 23:59 integration_phases.
-rw-r--r-- 1 milosvasic milosvasic  67426 May  4 23:59 overall_architectur
```

Canonical topology source: `docs/improvements/canonical/topology.yaml`

Regenerator script: `scripts/regenerate-diagrams.py` (executable; uses Agg backend for headless operation)

DEPRECATED.md pointers placed: - `docs/improvements/01_analysis_step_01/DEPRECATED.md` - `docs/improvements/02_analysis_step_02/DEPRECATED.md`

Final `make verify-foundation` **output (last 25 lines):**

```
./dependencies/HuggingFace_Hub/tests/test_hf_api.py:4187:    @pytest.mark.
./dependencies/HuggingFace_Hub/tests/test_hf_file_system.py:439:    @unitt
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:64:@py
```

```
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:73:@py
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:91:@py
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:105:@p
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:112:@p
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:132:@p
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:147:@p
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:165:@p
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:191:@p
./dependencies/HuggingFace_Hub/tests/test_inference_async_client.py:217:@p
... (3800 more — re-run 'scripts/no-silent-skips.sh' without head)

Annotate each with a trailing '// SKIP-OK: #<ticket>' (or '# SKIP-OK: #<ti
comment so the skip is tracked, or remove the skip if it is no longer need

(warn-only mode — set NO_SILENT_SKIPS_WARN_ONLY=0 to fail the build)
OK: no credential patterns found in .
FAIL: LLMsVerifier pin divergence
  submodules/llms_verifier  → d473231d27196e2151405f37936151a386b590e3
  helix_agent/LLMsVerifier → 1d53ae3b72c77c1f27171c0677431c48d2d02bdd

Resolution: pick the canonical SHA, bump the other to match, commit, push.
make: *** [Makefile:54: verify-llmsverifier-pin-parity] Error 1
```

verify-foundation exit code: 2 — LLMsVerifier dual-pin divergence (out-of-scope per spec §1.3 N2; documented as the carry-forward open item to Phase 1).

Phase 0 — CLOSED

Tasks completed: 16 plan tasks + 2 added during execution (T08.5, T08.7) + 1 deferred (T02 cosmetic) = 17 closed + 1 deferred + 1 polish-fix-it.

Final SHA on all 4 remotes: `3676f411073f1fa9ac4841eb184d3a6734231fd3` — all 4 remotes (github, gitlab, origin, upstream) converge at this SHA.

Open carry-forward items (Phase 1+): 1. **LLMsVerifier dual-pin divergence** — `submodules/llms_verifier` ahead of `helix_agent/LLMsVerifier`. Resolution requires HelixAgent-internal commit (out of scope per spec §1.3 N2). Phase 1 sub-spec for any feature that depends on LLMsVerifier behaviour must include the pin coordination. > **Resolution annotation (close-out[45], 2026-05-15):** RESOLVED in P1.5-WP2. Transitive duplicate eliminated; HelixAgent now consumes the canonical pin via `go.mod` replace at `../submodules/llms_verifier/llm-verifier`. `make verify-foundation` exits 0. Original divergence text preserved above as historical record per evidence-doc convention. 2. **3 historical credential leaks** — already remediated in P0-T08.5 (files removed from index, replaced with .example templates, ephemeral generation script for SSH keys). Required operator action (separate from this programme): rotate the SonarQube + Snyk tokens; reject the leaked SSH public key wherever trusted. 3. **13 cli_agents with stale HelixAgent pins** — `aider, conduit, continue, HelixCode, kilo-code, kiro-cli, mobile-agent, ollama-code, opencode-cli, openhands, plandex, roo-code, superset` will need their HelixAgent pin bumped before Phase 2 sub-specs touch them. 4. **Submodule recursion cosmetic error** — `Example_Projects/{Agent-Deck,Bridle,Claude-Code-Plugins-And-Skills}` cause `git submodule foreach --recursive` to fatal-out; modifying third-party submodules is forbidden. Scripts wrap with `|| true`.

Phase 1 unblocked: claude-code-source porting can proceed. The helix_agent/cli_agents/claude-code/ source is fully populated; the inner Go app (helix_code/) has its governance triplet; secret hygiene is in place; pre-push hook is installed; scan-secrets gates pre-push; SonarQube + Snyk infrastructure is wired (live scans pending operator credential rotation). | No third-party submodule modifications | ✓ |